

Retailer Data Subscriptions Design

Status: Proposed **Created:** 2026-02-28 **Related:** [specs/sponsored-placements-design.md](#), [specs/payout-design.md](#), [specs/affiliate-network-design.md](#)

Overview

Retailer data subscriptions give LYKE a predictable monthly recurring revenue stream. Retailers pay a monthly or annual fee to unlock tiered access to LYKE's body-profile and fit intelligence data — insights that are impossible to obtain anywhere else. This document covers the subscription tiers, feature gating, billing integration via Stripe Billing, the data model, and all required API and frontend changes.

Current State

What exists	Gap
GET /api/retailers/v1/portal/fit-insights — per-product fit data	No access control; all retailers get full data
GET /api/retailers/v1/portal/body-insights — body profile breakdown	No access control; all retailers get full data
GET /api/retailers/v1/portal/analytics — click/conversion analytics	No access control; all retailers get full data
<code>Retailer</code> entity with <code>AffiliateConfig</code> JSON	No <code>SubscriptionTier</code> field
No <code>RetailerSubscription</code> entity	No subscription lifecycle
<code>RetailerBillingAccount</code> (from specs/sponsored-placements-design.md)	Shared Stripe Customer — can be extended for Stripe Billing

Subscription Tiers

Tier Definitions

Tier	Price	Billing	Target customer
Starter	£0	—	New retailers; 30-day trial of Growth features, then permanent free tier
Growth	£299 / month	Monthly or annual (£2,990 = 2 months free)	Independent brands, small-to-mid retailers
Pro	£799 / month	Monthly or annual (£7,990 = 2 months free)	Mid-market retailers, multi-brand groups
Enterprise	Custom	Annual contract	Large retailers, department stores, retail groups

A new retailer automatically gets a **30-day Growth trial** when their account is approved. No payment method is required to start the trial. On day 30 the subscription downgrades to Starter unless they add a card.

Feature Matrix

Feature	Starter	Growth	Pro	Enterprise
Basic click & conversion analytics (rolling 30 days)	✓	✓	✓	✓
Extended analytics history (90 days)	—	✓	✓	✓
Extended analytics history (365 days)	—	—	✓	✓
Fit insights (per-product fit distribution + size breakdown)	—	✓	✓	✓
Body profile insights (anonymised height/weight/type bands)	—	—	✓	✓
Category affinity breakdown (which user segments browse which categories)	—	—	✓	✓
CSV/JSON export of insights data	—	—	✓	✓
Sponsored placements (campaign creation and billing)	✓	✓	✓	✓
Dedicated account manager	—	—	—	✓
Custom analytics SLA	—	—	—	✓

Sponsored placements are not gated by subscription tier. Campaign billing is a separate purchase flow that any retailer can use regardless of their subscription level.

Data Model Changes

New Enum: **SubscriptionTier**

```
public enum SubscriptionTier
{
    Starter      = 0,
    Growth       = 1,
    Pro          = 2,
    Enterprise   = 3
}
```

New Enum: **SubscriptionStatus**

```
public enum SubscriptionStatus
{
    Trialing      = 0,
    Active        = 1,
    PastDue       = 2,    // payment failed; grace period active
    CancelledAtPeriodEnd = 3, // will cancel at end of current period
}
```

```

    Cancelled      = 4,
    Paused         = 5    // used for Enterprise custom terms
}

```

New Enum: `BillingCycle`

```

public enum BillingCycle
{
    Monthly = 0,
    Annual  = 1
}

```

New Entity: `RetailerSubscription`

```

RetailerSubscription
├─ Id (GUID)
├─ RetailerId (FK → Retailer, unique)
├─ Tier (SubscriptionTier enum)
├─ Status (SubscriptionStatus enum)
├─ BillingCycle (BillingCycle enum)
├─ StripeSubscriptionId (string?, encrypted) ← null for Starter (no
Stripe subscription)
├─ StripePriceId (string?) ← the Stripe Price ID for
the current plan
├─ TrialEndsAt (DateTime?) ← set on creation for 30-
day Growth trial
├─ CurrentPeriodStart (DateTime?)
├─ CurrentPeriodEnd (DateTime?)
├─ CancelAtPeriodEnd (bool) ← mirrors Stripe field
├─ CreatedAt (DateTime)
└─ UpdatedAt (DateTime?)

```

`StripeSubscriptionId` is encrypted at rest using the same `IEncryptionService` used in `specs/payout-design.md`.

Modified: `Retailer`

Add a denormalised `SubscriptionTier` field directly on the `Retailer` entity for fast read access (avoids a join on every gated request):

```

Retailer
├─ ... (existing fields)
└─ SubscriptionTier (enum, default = Starter)

```

This is kept in sync with `RetailerSubscription.Tier` whenever the subscription changes. It is the authoritative field for feature gating at runtime. `RetailerSubscription` is the authoritative record for billing state and Stripe linkage.

Billing Infrastructure

Subscriptions reuse the `RetailerBillingAccount` defined in `specs/sponsored-placements-design.md`. The Stripe Customer is created once and shared across both campaign charges and subscriptions. No additional billing entity is needed.

Stripe Products and Prices

LYKE configures the following in the Stripe dashboard (not in code):

Stripe Product	Stripe Price (monthly)	Stripe Price (annual)
LYKE Growth	£299/month	£2,990/year
LYKE Pro	£799/month	£7,990/year

Price IDs are stored in `appsettings.json` under `RetailerSubscription:StripePriceIds`:

```
"RetailerSubscription": {
  "GrowthMonthlyPriceId": "price...",
  "GrowthAnnualPriceId": "price...",
  "ProMonthlyPriceId": "price...",
  "ProAnnualPriceId": "price...",
  "TrialDurationDays": 30
}
```

Enterprise subscriptions are managed manually (Stripe invoices, not subscriptions).

Subscription Lifecycle

New Retailer Flow

- 1. Retailer account approved
 - └─ `RetailerSubscription` created: `Tier = Growth`, `Status = Trialing`, `TrialEndsAt = now + 30 days`
 - └─ `Retailer.SubscriptionTier = Growth`
 - └─ No Stripe call (no payment method required for trial)
- 2. Day 30: Stripe trial-end webhook OR `TrialMonitor` background job fires
 - └─ If no payment method on `RetailerBillingAccount`:
 - └─ Downgrade to Starter (no Stripe subscription created)
 - └─ `Retailer.SubscriptionTier = Starter`
 - └─ Email: `trial-expired.liquid`
 - └─ If payment method saved:

- └ Stripe Subscription created; first invoice charged immediately
- └ RetailerSubscription.Status = Active
- └ Email: subscription-activated.liquid

Upgrade / Downgrade Flow

Retailer selects new tier → POST
/api/retailers/v1/portal/subscription/change

If upgrading (e.g. Growth → Pro):

- └ Stripe immediately invoices pro-rated difference
- └ Stripe Subscription updated (stripe.subscriptions.update with proration_behavior = "always_invoice")
- └ RetailerSubscription.Tier and Retailer.SubscriptionTier updated immediately
- └ Email: subscription-upgraded.liquid

If downgrading (e.g. Pro → Growth):

- └ Stripe schedules downgrade at end of current billing period
- └ RetailerSubscription.CancelAtPeriodEnd = true; new tier stored as pending
- └ At period end webhook: apply new tier
- └ Email: subscription-downgrade-scheduled.liquid

If cancelling to Starter:

- └ Same as downgrade; Stripe subscription cancelled at period end
- └ Email: subscription-cancellation-scheduled.liquid

Payment Failure Grace Period

invoice.payment_failed webhook received:

- └ RetailerSubscription.Status = PastDue
- └ Email: payment-failed.liquid (with link to update card)
- └ Stripe automatically retries (Smart Retries enabled in Stripe dashboard)

After 3 failed retries (Stripe sends customer.subscription.deleted):

- └ RetailerSubscription.Status = Cancelled, Tier = Starter
- └ Retailer.SubscriptionTier = Starter
- └ Email: subscription-lapsed.liquid

Feature Gating

ISubscriptionGate Service

```
public interface ISubscriptionGate
{
    /// <summary>Returns true if the retailer's current tier includes the
    given feature.</summary>
    bool IsFeatureAccessible(SubscriptionTier tier, RetailerFeature
    feature);

    /// <summary>Throws ForbiddenException with upgrade prompt if feature
    is not accessible.</summary>
    void RequireFeature(SubscriptionTier tier, RetailerFeature feature);
}
```

RetailerFeature Enum

```
public enum RetailerFeature
{
    BasicAnalytics           = 0,    // 30-day window - all tiers
    ExtendedAnalytics90      = 1,    // 90-day window - Growth+
    ExtendedAnalytics365     = 2,    // 365-day window - Pro+
    FitInsights              = 3,    // Growth+
    BodyProfileInsights      = 4,    // Pro+
    CategoryAffinityData     = 5,    // Pro+
    DataExport               = 6,    // Pro+
}
```

Gating at the Endpoint Layer

Feature checks happen in the endpoint handlers before calling any service. `Retailer.SubscriptionTier` is read from the authenticated retailer's profile (already loaded for auth checks) — no additional DB query.

Example:

```
app.MapGet("/api/retailers/v1/portal/fit-insights", async (...) =>
{
    _gate.RequireFeature(retailer.SubscriptionTier,
    RetailerFeature.FitInsights);
    // ... proceed
});
```

`RequireFeature` throws a `403 Forbidden` with:

```
{
  "success": false,
  "error": {
    "code": "SUBSCRIPTION_REQUIRED",
    "message": "Fit insights require a Growth subscription or higher.",
  }
}
```

```
    "details": { "requiredTier": "Growth", "currentTier": "Starter",
"upgradeUrl": "/retailer/billing/upgrade" }
  }
}
```

Analytics Date Window Enforcement

`RetailerAnalyticsRequest.StartDate` is validated against the retailer's tier in the service layer:

Tier	Maximum lookback
Starter	30 days
Growth	90 days
Pro / Enterprise	365 days

Requests exceeding the window are clamped (not rejected) to preserve UX — the response includes a `meta.windowClamped: true` flag so the frontend can show an upgrade prompt.

New API Endpoints

All new endpoints under `/api/retailers/v1/portal/subscription/` in `RetailerEndpoints.cs`.

```
GET    /api/retailers/v1/portal/subscription
  → Returns current RetailerSubscription state
  → Response: { tier, status, billingCycle, trialEndsAt, currentPeriodEnd,
cancelAtPeriodEnd }

POST   /api/retailers/v1/portal/subscription/change
  → Body: { tier: "Growth"|"Pro"|"Starter", billingCycle:
"Monthly"|"Annual" }
  → Handles upgrade, downgrade, and cancellation to Starter
  → Returns updated subscription state

GET    /api/retailers/v1/portal/subscription/portal
  → Returns a Stripe Billing Portal session URL (short-lived, redirect
immediately)
  → Retailer can view invoice history, update card, download receipts

POST   /api/webhooks/stripe (extend existing handler)
  → customer.subscription.updated → sync RetailerSubscription status and
tier
  → customer.subscription.deleted → cancel subscription, downgrade to
Starter
  → invoice.payment_succeeded → update CurrentPeriodStart/End
  → invoice.payment_failed → set PastDue, send email
```

New Services

IRetailerSubscriptionService (Application layer)

```
public interface IRetailerSubscriptionService
{
    Task<RetailerSubscription> GetSubscriptionAsync(Guid retailerId,
CancellationToken ct = default);
    Task<RetailerSubscription> ChangeSubscriptionAsync(Guid retailerId,
SubscriptionTier newTier, BillingCycle cycle, CancellationToken ct =
default);
    Task<string> GetBillingPortalUrlAsync(Guid retailerId, string
returnUrl, CancellationToken ct = default);

    // Webhook handlers
    Task HandleSubscriptionUpdatedAsync(string stripeSubscriptionId,
CancellationToken ct = default);
    Task HandleSubscriptionDeletedAsync(string stripeSubscriptionId,
CancellationToken ct = default);
    Task HandleInvoicePaymentSucceededAsync(string stripeSubscriptionId,
CancellationToken ct = default);
    Task HandleInvoicePaymentFailedAsync(string stripeSubscriptionId,
CancellationToken ct = default);
}
```

This service depends on `IRetailerBillingService` (from `sponsored-placements-design.md`) to obtain the `RetailerBillingAccount` and Stripe Customer ID.

Configuration

RetailerSubscriptionSettings

```
public class RetailerSubscriptionSettings
{
    public const string SectionName = "RetailerSubscription";

    public string GrowthMonthlyPriceId { get; set; } = string.Empty;
    public string GrowthAnnualPriceId { get; set; } = string.Empty;
    public string ProMonthlyPriceId { get; set; } = string.Empty;
    public string ProAnnualPriceId { get; set; } = string.Empty;

    // Duration of the free Growth trial for new retailers
    public int TrialDurationDays { get; set; } = 30;

    // Analytics lookback windows per tier (days)
    public int StarterAnalyticsWindowDays { get; set; } = 30;
    public int GrowthAnalyticsWindowDays { get; set; } = 90;
    public int ProAnalyticsWindowDays { get; set; } = 365;
}
```

Email Notifications

Event	Template	Recipient
Trial started (account approved)	<code>trial-started.liquid</code>	Retailer
Trial ending in 7 days (no card saved)	<code>trial-ending.liquid</code>	Retailer
Trial expired, downgraded to Starter	<code>trial-expired.liquid</code>	Retailer
Subscription activated (trial converted)	<code>subscription-activated.liquid</code>	Retailer
Subscription upgraded	<code>subscription-upgraded.liquid</code>	Retailer
Downgrade scheduled for period end	<code>subscription-downgrade-scheduled.liquid</code>	Retailer
Cancellation scheduled for period end	<code>subscription-cancellation-scheduled.liquid</code>	Retailer
Payment failed	<code>payment-failed.liquid</code>	Retailer
Subscription lapsed (all retries failed)	<code>subscription-lapsed.liquid</code>	Retailer
Annual renewal reminder (14 days before)	<code>subscription-renewal.liquid</code>	Retailer

The **trial-ending** email is sent by a scheduled `TrialExpiryReminderFunction` (Azure Functions timer trigger) that runs daily and finds trials expiring in 7 days with no saved payment method.

Frontend Changes

Retailer — Subscription Management Page (new)

- Route: `/retailer/billing/subscription`
- Shows: current tier, status, next billing date, trial countdown (if trialing)
- Tier comparison table with feature matrix and pricing
- Upgrade/downgrade CTAs — upgrades take effect immediately, downgrades scheduled
- "Manage billing" button → opens Stripe Billing Portal in browser tab

Retailer — Analytics Pages (update existing)

- When a retailer on Starter requests data beyond their window, show a banner: *"Data beyond 30 days requires a Growth subscription. [Upgrade →]"*
- When a Starter retailer visits the Fit Insights or Body Profile Insights pages, show a full-page upgrade prompt instead of the data table
- Upgrade prompt shows the relevant tier, price, and a CTA to the subscription management page

Retailer — Campaign Portal (update existing)

- Add a "subscription" menu item to the retailer portal nav
 - Show the current tier badge next to the retailer's name in the header (e.g. "Growth" chip)
-

GDPR Considerations

- Body profile insights data returned by `BodyProfileInsightsResponse` is already anonymised and aggregated — only band-level counts and percentages, never individual profiles.
 - `MinimumGroupSize` on `BodyProfileInsightsResponse` (currently hardcoded) is enforced in `RetailerService` to suppress bands with fewer than 5 users. This applies regardless of subscription tier.
 - Subscription and billing data (Stripe IDs, last 4 digits) are classed as financial data — encrypted at rest. Retailers can request deletion of their account, which triggers Stripe customer deletion and zeroing of all personal fields per existing GDPR deletion logic.
-

Open Questions

1. **Enterprise billing** — Enterprise retailers pay via annual invoice rather than Stripe Subscription. For MVP, LYKE manually sets `RetailerSubscription.Tier = Enterprise` and `Status = Active` via an admin endpoint with `CurrentPeriodEnd` set to the contract expiry. A proper invoice flow is a Phase 2 item.
2. **Trial without a payment method** — the Growth trial requires no card, which means there is no Stripe Subscription object during the trial period. The `TrialExpiryReminderFunction` handles the conversion. Alternatively, Stripe supports free trials on subscriptions (`trial_period_days`) which would unify the lifecycle into Stripe; this is cleaner but requires collecting a card upfront, which may reduce trial sign-up rates.
3. **Pausing subscriptions** — Pro/Enterprise retailers who go offline temporarily (e.g. seasonal brands) may want to pause billing. Stripe supports subscription pausing. Defer to a later phase; `SubscriptionStatus.Paused` is reserved in the enum.
4. **Annual renewal emails** — the 14-day renewal reminder requires knowing each retailer's renewal date. This is available via `CurrentPeriodEnd`. A `AnnualRenewalReminderFunction` timer trigger handles this identically to the trial reminder.